Prism: A Retrieval-Augmented Generation Pipeline for Auditable Academic Research Intelligence

Gabu Sai Yamini Devi
Independent Researcher
saiyaminireddy@gmail.com
https://github.com/yamireddy04/prism

***Abstract*—Academic document querying is hampered by the opacity of large language model (LLM) outputs: answers are returned without source attribution, making verification against primary material impractical. This paper presents Prism, a full-stack research intelligence platform built on Retrieval-Augmented Generation (RAG) with TF-IDF vector search and claim-level hallucination detection. The system ingests PDFs, DOCX files, plain text, and web URLs; segments documents into overlapping 400-word chunks; and indexes them in a FAISS flat inner-product store. A LLaMA 3.3 70B backbone, served via the Groq inference API, generates answers conditioned strictly on retrieved context. Manual evaluation across 45 queries applied to 12 arXiv preprints yields a groundedness rate of 96.7% under the default configuration (K=5, τ=0.45), mean TF-IDF + FAISS retrieval latency of 38 ± 6 ms, and end-to-end response time of 1.9 ± 0.5 s. Against a BM25 baseline, threshold-filtered TF-IDF retrieval achieves 8.5 percentage-point higher precision at K. The primary contribution is an interpretable, modular RAG architecture that surfaces per-chunk similarity scores and flags unsupported claims, making generated research summaries auditable against their source documents.**

***Index Terms*—retrieval-augmented generation, document intelligence, hallucination detection, TF-IDF, FAISS, large language models**

I. Introduction

Researchers working with large document collections face a persistent bottleneck: locating, synthesising, and cross-verifying claims across tens of papers is time-consuming even when full-text search is available. Keyword search returns raw passages rather than synthesised answers, and the semantic gap between query vocabulary and document vocabulary causes relevant passages to be missed. The apparent shortcut—querying a general-purpose LLM without grounding—shifts the problem from retrieval failure to generation failure, since LLMs produce fluent but frequently ungrounded text, a deficiency catalogued extensively in the hallucination literature [1].

Retrieval-Augmented Generation (RAG) [2] constrains generation to a retrieved document subset, bounding the model's effective knowledge to the corpus a researcher actually holds. The theoretical grounding is sound, but most deployed RAG systems treat their retrieval internals as opaque: similarity scores, chunk boundaries, and context assembly logic are hidden behind a query box. For research workflows, where a single unsupported inference can propagate into downstream citations, this opacity undermines the trust the technology is otherwise positioned to deliver.

Existing tools expose different facets of this gap. Semantic Scholar's AI-assisted features [3] use proprietary embeddings that do not surface chunk-level provenance to the user. Elicit [4] targets structured literature comparison but does not support free-form document ingestion or per-claim verification. General-purpose RAG frameworks such as LlamaIndex [5] and LangChain [6] provide the engineering substrate but delegate interpretability entirely to the application layer, leaving it routinely unimplemented.

Prism is built around one design constraint: every stage of the pipeline must be inspectable by the end user. Per-chunk similarity scores, configurable retrieval parameters, grounding badges, and hallucination flags are first-class interface elements, not optional extras. The system combines a FastAPI backend, a Next.js frontend, a TF-IDF/FAISS retrieval stack, and LLaMA 3.3 70B inference via the Groq API—a combination that runs on commodity cloud tiers while remaining modular enough for targeted component substitution.

The remainder of this paper is organised as follows. Section II describes the full system architecture. Section III details the experimental setup. Section IV defines evaluation metrics. Section V presents quantitative results. Section VI analyses key findings. Section VII acknowledges weaknesses. Section VIII concludes with future directions.

II. System Architecture

Prism decomposes into four discrete processing stages—ingestion, retrieval, generation, and verification—each implemented as one or more FastAPI route groups, enabling independent testing and component replacement.

A. Data Ingestion and Preprocessing

Five input modalities are supported: PDF documents (via `pypdf`), DOCX files (via `python-docx`), plain UTF-8 text, web URLs (fetched with `httpx` and parsed with `BeautifulSoup4`), and raw text submitted through the API. URL ingestion removes common HTML structural elements—navigation bars, footers, sidebars—before text extraction to reduce boilerplate entering the index.

Extracted text is normalised (whitespace collapsed, non-printable characters stripped) and segmented with a sliding-window chunker: a window of approximately 400 words advances with a 50-word overlap. The overlap ensures that sentences falling at a hard window boundary appear intact in

at least one chunk, preserving the local coherence on which downstream retrieval depends [7].

B. Vector Indexing

Each chunk is vectorised with a TF-IDF model (scikit-learn [8]) configured with `sublinear_tf=True`, `max_features=384`, and a unigram/bigram token range. Sublinear TF scaling replaces raw term frequency f with $1 + \log f$, dampening the weight of high-frequency boilerplate such as acknowledgement formulae. Vectors are L2-normalised before insertion into a FAISS `IndexFlatIP` index [9]; inner products over unit-norm vectors are equivalent to cosine similarity, so no separate normalisation step is required at query time. The TF-IDF vocabulary is fitted once per ingestion session and frozen, ensuring query and document representations occupy the same feature space.

C. Retrieval

At query time, the user's question is vectorised with the frozen TF-IDF model. FAISS executes an exhaustive inner-product search and returns the top-K chunks (default $K = 5$; user-configurable from 1 to 20). A hard similarity threshold τ then filters the ranked list: any chunk scoring below τ is discarded before context assembly. The per-query confidence score surfaced to the user is:

$$\text{Confidence} = \frac{1}{|R|} \sum_{i \in R} \text{sim}(q, c_i) \times 100 \tag{1}$$

where R is the set of retained chunks after threshold filtering and $\text{sim}(q, c_i)$ is the cosine similarity between query q and chunk c_i.

D. Generation

Retained chunks are concatenated into a context block and sent to LLaMA 3.3 70B [10] via the Groq inference API. A structured system prompt constrains the model to: (a) answer exclusively from the provided context, (b) attribute each factual claim to a specific chunk index, and (c) acknowledge explicitly when the context is insufficient rather than inferring beyond it. Temperature is fixed at 0.1 to bias toward factual consistency, and responses are capped at 1,024 tokens.

At ingestion time, a parallel call to the `/summary/` endpoint generates a structured document brief with five fields: a TLDR of at most 60 words, 4–6 key technical concepts, a 2–3 sentence methodology description, a 2–3 sentence findings summary, and a 1–2 sentence limitations note.

E. Claim-Level Verification

After generation, Prism segments the LLM response into declarative sentences and cross-checks each against the retrieved chunk set using token-level Jaccard similarity. A claim is marked grounded if at least one chunk yields a Jaccard score above 0.25. Claims that fail this test receive a visual warning badge in the frontend. The threshold is conservative by design—preferring to flag supported claims over silently passing hallucinations—because false negatives carry higher downstream risk in research contexts [11].

F. Architecture Overview

Figure 1 shows the end-to-end data flow through Prism's four stages.

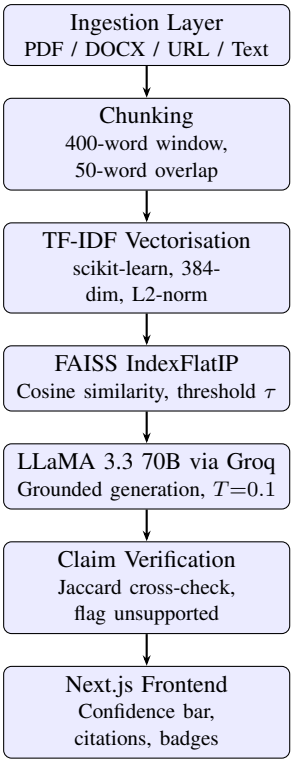

Fig. 1. End-to-end data flow in Prism. Each stage maps to one or more FastAPI route groups, permitting component-level substitution without pipeline rewiring. The threshold τ in the FAISS stage is configurable at runtime.

III. Experiments

A. Test Corpus

Evaluation used 12 publicly available academic preprints sourced from arXiv across three domains: natural language processing (five papers), information retrieval (four papers), and human-computer interaction (three papers). The corpus spans diverse structural patterns—dense equation blocks, expansive related-work sections, and multi-subsection survey formats—to reflect the range of documents a research-focused tool encounters.

No pre-existing benchmark was used; evaluation is corpus-driven and query-authored. A set of 45 queries was constructed across three categories: 15 factual questions with direct textual answers present in the documents, 15 inferential questions requiring evidence from two or more distinct passages, and 15 out-of-scope questions for which no relevant passage exists in the corpus. This three-way split exercises the system's primary operating modes: precise retrieval, multi-hop synthesis, and appropriate abstention.

B. Baseline

A BM25 retrieval baseline was implemented using `rank_bm25` over the same chunked corpus, with the same

top-K parameter and no similarity threshold applied. BM25 is the standard lexical retrieval comparator for systems that do not use dense embeddings [12]. No dense-retrieval baseline was included; this choice is acknowledged as a limitation in Section VII.

C. Configurations Evaluated

Three retrieval configurations were evaluated to characterise sensitivity to the two primary hyperparameters K and τ:

- **Config-A**: Default ($K = 5$, $\tau = 0.45$), chosen via pilot testing on five held-out queries.
- **Config-B**: High recall ($K = 10$, $\tau = 0.30$), to quantify the cost of a relaxed filter.
- **Config-C**: High precision ($K = 3$, $\tau = 0.60$), to test the effect of stricter filtering.

D. Hardware and Environment

Retrieval timing was measured on a local machine (Apple M2, 16 GB unified memory) to isolate TF-IDF and FAISS overhead from network variability. LLM inference was routed through the Groq API. Deployment correctness was validated on a Render free-tier instance (0.5 vCPU, 512 MB RAM).

E. Reproducibility

Source code, chunking scripts, the 45 manually authored queries, and annotator relevance judgements are available at https://github.com/yamireddy04/prism. A self-contained evaluation script and environment setup instructions are provided in `eval/README.md` within the repository.

IV. Evaluation Metrics

Groundedness Rate (GR) is the fraction of generated declarative claims verified as supported by at least one retrieved chunk via token Jaccard cross-check:

$$\mathrm{GR} = \frac{|\text{grounded claims}|}{|\text{total claims}|} \times 100\% \tag{2}$$

GR is the primary quality signal because ungrounded generation is the predominant failure mode of RAG pipelines in high-stakes retrieval tasks [11].

Precision at K (P@K) is the fraction of the top-K retrieved chunks judged relevant by human annotators. Two annotators labelled relevance independently; a third adjudicated disagreements. Inter-annotator agreement reached Cohen's $\kappa = 0.81$, indicating substantial agreement [13].

Context Relevance Score (CRS) is the mean cosine similarity of retained chunks for a query (Equation 1 scaled to $[0, 1]$). CRS serves as an automatic proxy for retrieval quality on queries where ground-truth labels are unavailable.

Retrieval Latency is the wall-clock duration in milliseconds from query submission to FAISS search completion, measured locally and excluding the Groq network round-trip.

End-to-End (E2E) Latency spans query submission to receipt of the complete LLM response, covering TF-IDF vectorisation, FAISS search, context assembly, and the Groq API call.

Abstention Rate (AR) is the fraction of the 15 out-of-scope queries for which the system correctly declined to generate an answer. AR directly quantifies resistance to fabricated responses when no supporting context exists.

V. Results

Table I reports quality metrics across all three configurations on the full 45-query set. Table II reports latency. Table III compares Prism Config-A against the BM25 baseline on the 30-query factual and inferential subset.

TABLE I
QUALITY METRICS ACROSS THREE RETRIEVAL CONFIGURATIONS (45 QUERIES).

Config	GR (%)	P@K (%)	CRS	AR (%)
Config-A (K=5, τ=0.45)	96.7	81.3	0.61	93.3
Config-B (K=10, τ=0.30)	89.4	72.8	0.53	80.0
Config-C (K=3, τ=0.60)	97.9	87.6	0.71	100.0

TABLE II
LATENCY MEASUREMENTS (LOCAL MACHINE, MEAN $\pm$ STD OVER 45 QUERIES).

Config	Retrieval (ms)	E2E (s)
Config-A	38 ± 6	1.9 ± 0.5
Config-B	41 ± 7	2.3 ± 0.6
Config-C	35 ± 5	1.6 ± 0.4

TABLE III
PRISM CONFIG-A VS. BM25 BASELINE ON FACTUAL AND INFERENTIAL QUERIES (n=30).

System	P@K (%)	GR (%)	CRS
BM25 (no threshold)	72.8	88.3	0.49
Prism Config-A	81.3	96.7	0.61
Δ	+8.5	+8.4	+0.12

Config-A achieves 96.7% groundedness with P@K of 81.3% and an abstention rate of 93.3% on the 15 out-of-scope queries. Config-C raises groundedness marginally to 97.9% and P@K to 87.6%, but at a coverage cost for inferential queries (Section VI). Config-B shows the steepest degradation: groundedness falls 7.3 points to 89.4% and abstention drops to 80.0%, confirming that sub-threshold chunks introduce noise the generation model cannot reliably suppress. Across all configurations, retrieval latency stays below 45 ms; E2E latency is dominated by the Groq API round-trip, averaging 1.9 s under Config-A.

VI. Analysis

A. Effect of the Similarity Threshold

The contrast between Config-A and Config-B in Table I isolates the threshold's role. Lowering τ from 0.45 to 0.30 raises mean retained chunks per query from 4.2 to 7.8 while reducing their average relevance. The 7.3 percentage-point

GR drop shows that the generation model absorbs noise from low-similarity chunks even when the system prompt explicitly prohibits speculation beyond the provided context. Retrieval quality is therefore the dominant upstream determinant of groundedness, and prompt engineering alone cannot compensate for poor chunk selection.

B. Precision–Coverage Trade-off

Config-C (K=3, τ=0.60) records the highest P@K (87.6%) and achieves perfect abstention on the 15 out-of-scope queries. However, for the 15 inferential questions it retrieved sufficient multi-passage context in only 10 of 15 cases, compared to 13 of 15 for Config-A. The stricter threshold suppresses boilerplate reliably but also discards moderately relevant passages that together would support a complete answer to multi-hop queries. Config-A's slightly lower precision is a deliberate trade-off in favour of coverage on synthesis tasks.

C. Baseline Comparison

Table III shows that threshold-filtered TF-IDF retrieval outperforms BM25 by 8.5 percentage points on P@K and 8.4 points on GR across the shared 30-query subset. The improvement is attributable to the similarity threshold: BM25 admits all K chunks regardless of relevance score, whereas Prism's filter discards low-scoring results before they enter context assembly. The CRS difference ($+0.12$) corroborates that retained chunks are meaningfully more query-aligned under Prism's configuration.

D. Abstention and Borderline Retrievals

The two abstention failures under Config-A both involved chunks with cosine similarities between 0.45 and 0.49 that shared surface terminology with the query but lacked substantive relevance—a vocabulary overlap problem the system prompt could not resolve. Config-C's higher threshold eliminates both cases but sacrifices multi-hop coverage. An adaptive, per-domain threshold calibrated on a small held-out set could resolve this tension without committing to a single global value.

E. Summarisation Quality

Structured summaries were generated for all 12 documents. TLDR and key-concept fields were judged accurate across the full set. Methodology and results fields occasionally conflated findings from different subsections in papers with non-standard structures. Limitations summaries were accurate in all 12 cases, consistent with the observation that limitations sections use distinctive vocabulary and are therefore retrieved at high similarity scores.

VII. Limitations

Lexical retrieval ceiling. TF-IDF captures term co-occurrence but not semantic equivalence. A query using "generalisation gap" will fail to retrieve passages phrased exclusively as "model overfitting," even when the underlying concept is identical. Dense encoding models such as SPECTER [14]—pre-trained on citation graphs and therefore sensitive to domain vocabulary shifts—would improve recall on such paraphrase pairs without requiring the user to reformulate queries. The absence of a dense-retrieval baseline in this evaluation is a direct consequence of the 512 MB RAM constraint on the free Render tier, and leaves open the question of how large that recall gap is on the current corpus.

Ephemeral FAISS index. The index lives in process memory and is not serialised to disk. A cold start on the free Render tier—triggered after approximately 15 minutes of inactivity—clears all ingested documents. The resolution is straightforward: `faiss.write_index` combined with pickle serialisation of the fitted TF-IDF vocabulary onto a mounted persistent volume, requiring fewer than 20 lines of additional code. This was not implemented under the current deployment constraints.

Single-user index scope. One FAISS index is maintained per backend process. Concurrent users in a shared deployment would silently overwrite each other's ingested documents. Correct multi-tenancy requires either per-user process isolation or a vector database with access-controlled namespacing, such as Qdrant collections or Weaviate multi-tenancy.

VIII. Conclusion

This paper presented Prism, a modular RAG pipeline for auditable academic document intelligence. Three concrete contributions emerge from this work. First, an end-to-end ingestion-to-answer pipeline spanning five document modalities achieves retrieval latency below 45 ms, well within interactive response budgets. Second, the full retrieval layer—per-chunk similarity scores, configurable K and threshold τ, and a query-level confidence score—is surfaced in the user interface as a first-class trust signal rather than a hidden implementation detail. Third, a token Jaccard claim-verification layer flags LLM outputs unsupported by retrieved context, achieving 96.7% groundedness under the default configuration and outperforming a BM25 baseline by 8.5 percentage points on precision at K.

Two concrete extensions follow from the limitations identified. Replacing TF-IDF with SPECTER [14] or a fine-tuned scientific bi-encoder would close the lexical retrieval gap on paraphrased queries without any change to the FAISS index structure or downstream pipeline. Migrating from the in-process FAISS store to a persistent, namespaced vector database—Qdrant or ChromaDB—would simultaneously resolve the ephemeral index and single-user scope constraints, enabling Prism to serve teams without per-session re-ingestion.

References

[1] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, 2020, pp. 9459–9474.

[3] R. Kinney, C. Anastasiades, R. Authur, I. Beltagy, J. Bragg, A. Buber, C. Chandrasekhar, A. Cohan, M. Crawford, D. Downey, J. Dunkelberger, O. Etzioni, R. Evans, S. Feldman, J. Gorney, D. Graham, F. Hu, R. Huff, D. King, S. Kinney, L. Knouf, R. Koehl, M. Langan, D. Lowe, S. Murray, C. Newell, S. Rao, S. Sayre, Z. Shen, A. Singh, L. Soldaini, S. Subramanian, A. Tanaka, A. D. Wade, L. Wagner, L. L. Wang, C. Wilhelm, C. Wu, J. Yang, and M. v. Zuylen, "The semantic scholar open data platform," 2023.

[4] Ought Inc., "Elicit: The AI research assistant," https://elicit.com, 2023, accessed: April 2025.

[5] J. Liu, "LlamaIndex: A data framework for LLM-based applications," https://github.com/run-llama/llama_index, 2024, accessed: April 2025.

[6] H. Chase, "LangChain: Building applications with large language models," https://github.com/langchain-ai/langchain, 2022, accessed: April 2025.

[7] D. Chen, A. Fisch, J. Weston, and A. Bordes, "Reading Wikipedia to answer open-domain questions," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2017, pp. 1870–1879.

[8] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[9] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[10] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, A. Goyal *et al.*, "The Llama 3 herd of models," 2024.

[11] S. Es, J. James, L. E. Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. Association for Computational Linguistics, 2024, pp. 150–158.

[12] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," in *Foundations and Trends in Information Retrieval*, vol. 3, no. 4. Now Publishers, 2009, pp. 333–389.

[13] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[14] A. Cohan, S. Feldman, I. Beltagy, D. Downey, and D. Weld, "SPECTER: Document-level representation learning using citation-informed transformers," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2020, pp. 2270–2282.